

Project: Users can edit,delete blogs with login.

Description:

This project is a simple blog management web application developed using **HTML, CSS, and JavaScript**. It allows users to **sign up and log in** securely, create new blog posts, **edit or delete their own blogs**, and view added content dynamically. All user interactions, validations, and blog operations are handled on the client side using JavaScript, while HTML provides structure, and CSS ensures a responsive and user-friendly interface. The project demonstrates core concepts of front-end web development, including DOM manipulation, form validation, and browser storage.

HTML file(index.html):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Blog App with Auth</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<!-- AUTH PAGE -->
<div id="authPage" class="glass">
  <h2 id="formTitle">Login</h2>

  <input type="text" id="username" placeholder="Username">
  <input type="password" id="password" placeholder="Password">

  <button onclick="handleAuth()">Login</button>

  <p id="toggleText">
    Don't have an account?
    <span onclick="toggleForm()">Sign up</span>
  </p>
```

```

    <p id="authError"></p>
</div>

<!-- BLOG PAGE -->
<div id="blogPage" class="glass hidden">
    <h2>My Blog</h2>
    <button class="logout-btn" onclick="logout()">Logout</button>

    <h3>Create Post</h3>
    <input type="text" id="title" placeholder="Title">
    <textarea id="content" placeholder="Write your blog..."></textarea>
    <button onclick="addPost()">Add Post</button>

    <h3>Posts</h3>
    <div id="posts"></div>
</div>

<script src="script.js"></script>
</body>
</html>

```

CSS file (style.css):

```

/* Reset */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

/* Background */
body {
    font-family: 'Segoe UI', sans-serif;
    min-height: 100vh;
    padding: 20px;

    background: linear-gradient(135deg, #667eea, #764ba2, #6dd5ed);
}

```

```

background-size: 300% 300%;
animation: bg 12s ease infinite;
}

/* Background animation */
@keyframes bg {
  0% { background-position: 0% 50%; }
  50% { background-position: 100% 50%; }
  100% { background-position: 0% 50%; }
}

/* Glass box */
.glass {
  max-width: 650px;
  margin: 50px auto;
  padding: 25px;

  background: rgba(255,255,255,0.12);
  backdrop-filter: blur(16px);
  -webkit-backdrop-filter: blur(16px);

  border-radius: 16px;
  border: 1px solid rgba(255,255,255,0.25);

  box-shadow: 0 8px 32px rgba(0,0,0,0.25);
  color: #fff;
}

/* Headings */
h2, h3 {
  text-align: center;
  margin-bottom: 15px;
}

/* Inputs */
input, textarea {
  width: 100%;
  padding: 12px;
  margin-top: 10px;

  border-radius: 10px;
  border: 1px solid rgba(255,255,255,0.3);
}

```

```
background: rgba(255,255,255,0.15);
color: #fff;

outline: none;
}

textarea {
  resize: vertical;
  min-height: 100px;
  max-height: 250px;
}

/* Buttons */
button {
  margin-top: 12px;
  padding: 10px 14px;

  border: none;
  border-radius: 10px;

  background: rgba(255,255,255,0.25);
  color: white;
  cursor: pointer;

  transition: 0.2s;
}

button:hover {
  background: rgba(255,255,255,0.4);
  transform: translateY(-2px);
}

/* Logout */
.logout-btn {
  float: right;
  background: rgba(255,80,80,0.5);
}

/* Posts */
.post {
  margin-top: 15px;
```

```
padding: 15px;

border-radius: 12px;
background: rgba(255,255,255,0.15);
border: 1px solid rgba(255,255,255,0.25);
}

.post p {
margin-top: 5px;
opacity: 0.9;
}

/* Post buttons */
.post button {
margin-right: 5px;
padding: 6px 10px;
font-size: 12px;
}

.edit-btn {
background: rgba(16,185,129,0.6);
}

.delete-btn {
background: rgba(239,68,68,0.6);
}

/* Toggle text */
#toggleText {
margin-top: 10px;
text-align: center;
}

#toggleText span {
cursor: pointer;
color: #fff;
font-weight: bold;
text-decoration: underline;
}

/* Hidden */
.hidden {
display: none;
}
```

```

}

/* Error */
#authError {
  margin-top: 10px;
  text-align: center;
  color: #ffdddd;
}

```

Script file (Script.js):

```
let isLogin = true;
```

```
// USERS + BLOG DATABASE
```

```
let users = JSON.parse(localStorage.getItem("users")) || [];
```

```
let postsDB = JSON.parse(localStorage.getItem("postsDB")) || {};
```

```
let currentUser = localStorage.getItem("currentUser");
```

```
// Toggle login/signup
```

```
function toggleForm() {
```

```
  isLogin = !isLogin;
```

```
  document.getElementById("formTitle").innerText = isLogin ? "Login" : "Sign Up";
```

```
  document.querySelector("#authPage button").innerText = isLogin ? "Login" : "Sign Up";
```

```
  document.getElementById("toggleText").innerHTML = isLogin
```

```
    ? ` Don't have an account? <span onclick="toggleForm()">Sign up</span> `
```

```
    : ` Already have an account? <span onclick="toggleForm()">Login</span> ` ;
```

```
}
```

```
// AUTH
```

```
function handleAuth() {
```

```
  const username = document.getElementById("username").value.trim();
```

```

const password = document.getElementById("password").value.trim();

if (!username || !password) return;

if (isLogin) {
  const user = users.find(u => u.username === username && u.password === password);

  if (user) {
    localStorage.setItem("currentUser", username);
    currentUser = username;
    showBlog();
  } else {
    showError("Invalid login");
  }
} else {
  const exists = users.find(u => u.username === username);

  if (exists) {
    showError("User already exists");
    return;
  }

  users.push({ username, password });
  localStorage.setItem("users", JSON.stringify(users));

  showError("Account created! Please login.");
  toggleForm();
}
}

// SHOW BLOG
function showBlog() {
  document.getElementById("authPage").classList.add("hidden");
  document.getElementById("blogPage").classList.remove("hidden");
  displayPosts();
}

// LOGOUT
function logout() {

```

```
localStorage.removeItem("currentUser");
currentUser = null;

document.getElementById("blogPage").classList.add("hidden");
document.getElementById("authPage").classList.remove("hidden");
}

// ERROR
function showError(msg) {
  document.getElementById("authError").innerText = msg;
}

// GET USER POSTS
function getUserPosts() {
  if (!postsDB[currentUser]) {
    postsDB[currentUser] = [];
  }
  return postsDB[currentUser];
}

// SAVE DB
function saveDB() {
  localStorage.setItem("postsDB", JSON.stringify(postsDB));
}

// ADD POST
function addPost() {
  const title = document.getElementById("title").value.trim();
  const content = document.getElementById("content").value.trim();

  if (!title || !content) return;

  let userPosts = getUserPosts();

  userPosts.push({ title, content });

  postsDB[currentUser] = userPosts;

  saveDB();
}
```



```

displayPosts();

document.getElementById("title").value = "";
document.getElementById("content").value = "";
}

// DISPLAY POSTS
function displayPosts() {
  const container = document.getElementById("posts");
  container.innerHTML = "";

  let userPosts = getUserPosts();

  userPosts.forEach((post, index) => {
    const div = document.createElement("div");
    div.className = "post";

    div.innerHTML = `
      <h4>${post.title}</h4>
      <p>${post.content}</p>
      <button class="edit-btn" onclick="editPost(${index})">Edit</button>
      <button class="delete-btn" onclick="deletePost(${index})">Delete</button>
    `;

    container.appendChild(div);
  });
}

// DELETE
function deletePost(index) {
  let userPosts = getUserPosts();

  userPosts.splice(index, 1);

  postsDB[currentUser] = userPosts;

  saveDB();
  displayPosts();
}

```

```
// EDIT
function editPost(index) {
  let userPosts = getUserPosts();

  const newTitle = prompt("Edit title:", userPosts[index].title);
  const newContent = prompt("Edit content:", userPosts[index].content);

  if (newTitle && newContent) {
    userPosts[index] = {
      title: newTitle,
      content: newContent
    };

    postsDB[currentUser] = userPosts;

    saveDB();
    displayPosts();
  }
}
```

Results:

